1. Fundamentos del proyecto – Ultima Transmisión

Estudiantes: Brayan Maluenga – Sebastian Ruiz

Proyecto: La última transmisión
Tecnología principal: Python
Formato del contenido narrativo: JSON
Control de versiones: Git y GitHub
Gestión del trabajo: Trello con metodología Kanban

1.1. Problema identificado

El proyecto busca ofrecer una experiencia narrativa interactiva en la que el jugador pueda tomar decisiones que afecten el desarrollo de la historia. Para ello, el sistema debe interpretar escenas, mostrar las opciones disponibles, evaluar condiciones, modificar variables y conducir al jugador hacia diferentes finales.

Una implementación en la que cada historia estuviera escrita directamente dentro del código dificultaría su modificación y reutilización. Por esta razón, se propone separar el contenido narrativo de las reglas del motor, permitiendo incorporar nuevas historias mediante archivos JSON sin modificar la lógica principal.

1.2. Actores del sistema

- **Jugador:** inicia una partida, lee las escenas, selecciona opciones, observa sus consecuencias, deshace su última decisión válida cuando corresponda y reinicia o finaliza la sesión.
- **Autor o responsable del contenido:** crea y modifica historias en formato JSON, define escenas, opciones, condiciones, efectos y finales.
- **Responsable de desarrollo y pruebas:** mantiene el motor narrativo, valida los archivos de historias, corrige defectos y ejecuta las pruebas automatizadas.

En esta entrega, una misma persona puede asumir las responsabilidades de autoría, desarrollo y pruebas.

1.3. Alcance

Incluido

- Cargar historias desde archivos JSON.
- Validar la estructura de las historias antes de iniciar una partida.
- Mostrar escenas y opciones disponibles.
- Evaluar condiciones booleanas para habilitar opciones.
- Aplicar efectos que modifican variables del estado narrativo.
- Permitir diferentes finales según las decisiones.

- Deshacer la última decisión válida y restaurar el estado anterior.
- Reiniciar una partida.
- Ejecutar pruebas automatizadas.
- Cargar una segunda historia utilizando el mismo motor.

Fuera del alcance

- Interfaz gráfica avanzada o aplicación web.
- Almacenamiento de perfiles de jugadores.
- Guardado permanente de partidas entre ejecuciones.
- Autenticación de usuarios.
- Multijugador y sincronización en línea.

1.4. Riesgos del proyecto

Riesgo	Probabilidad	Impacto	Mitigación
Una historia contiene referencias a escenas inexistentes o variables mal definidas.	Media	Alto	Validar el JSON antes de iniciar la partida y mantener pruebas para estructuras inválidas.
Un cambio en el motor altera comportamientos que antes funcionaban, como los finales o la función de deshacer.	Media	Alto	Ejecutar la suite de regresión antes de integrar y entregar cada cambio.

La probabilidad indicada es una estimación inicial del equipo y debe revisarse cuando aparezca nueva evidencia.

1.5. Priorización de atributos de calidad

Se priorizan los siguientes atributos para esta entrega:

1. **Correctitud funcional:** las opciones, condiciones, efectos, finales y operaciones de deshacer deben producir el comportamiento especificado.
2. **Mantenibilidad:** el contenido narrativo debe poder modificarse sin alterar el motor.
3. **Fiabilidad:** los archivos con errores deben generar mensajes comprensibles en lugar de iniciar una partida con datos inválidos.
4. **Facilidad de uso:** el jugador debe entender cómo elegir una opción, deshacer, reiniciar y salir.

El orden de prioridad responde al alcance académico: primero debe funcionar correctamente el comportamiento narrativo; después, facilitar su evolución y uso

1.6. Responsabilidad profesional

Se mantendrá la autoría mediante el historial de Git, con mensajes de commit descriptivos y evidencia de los cambios realizados. Los materiales externos y las herramientas de inteligencia artificial que se utilicen se identificarán en la entrega, indicando para qué se emplearon y cómo se verificaron sus resultados.

El contenido de las historias deberá ser original o contar con la atribución correspondiente. No se incorporarán datos personales reales que no sean necesarios para el juego. Las dependencias se mantendrán limitadas a las necesarias para el proyecto, y se comprobará si alguna biblioteca externa requiere atribución o condiciones de licencia específicas.

La responsabilidad final sobre el código, los requisitos, las pruebas y los materiales entregados corresponde a quienes presentan el proyecto; las herramientas de asistencia no sustituyen su revisión técnica.

2. Proceso de desarrollo

Para el desarrollo de La última transmisión se seleccionó un proceso incremental apoyado en la metodología Kanban. Este enfoque permite construir el sistema mediante incrementos funcionales, validar cada avance y actualizar los artefactos del proyecto conforme se incorporan cambios.

Se consideró como alternativa el modelo en cascada, que organiza el desarrollo en fases secuenciales. Aunque este modelo puede ser apropiado cuando los requisitos permanecen estables, se decidió no utilizarlo como enfoque principal porque el proyecto requiere implementar funcionalidades, corregir defectos y comprobar continuamente el comportamiento del motor narrativo. El enfoque incremental facilita responder a estos cambios y detectar regresiones antes de la entrega.

El proceso se aplica mediante cuatro actividades. **Especificación:** se definen los requerimientos funcionales y no funcionales, los criterios de aceptación y las relaciones entre requisitos, componentes y pruebas. **Desarrollo:** se implementan las funcionalidades por incrementos en Python, utilizando archivos JSON para mantener separado el contenido narrativo de las reglas del motor y Git para registrar las modificaciones. **Validación:** se ejecutan pruebas automatizadas y pruebas de comportamiento para comprobar las condiciones, los efectos, los finales, el reinicio, la función de deshacer y el rechazo de historias inválidas. **Evolución:** se incorporan cambios y correcciones, se comparan las versiones del código y se actualizan las pruebas, los requisitos y la documentación de trazabilidad.

Kanban complementa este proceso al permitir visualizar las tareas en los estados Backlog, Por hacer, En progreso, En revisión y pruebas, y Hecho. Una tarea solo se considera terminada cuando su implementación ha sido verificada, las pruebas pertinentes han sido ejecutadas y la documentación correspondiente se encuentra actualizada.

Las principales evidencias de este proceso serán el historial de commits de Git, el tablero Kanban con sus capturas inicial y final, los archivos de pruebas, la documentación de los requisitos y la matriz de trazabilidad.

3. Requerimientos

3.1. Requisitos funcionales

Los requisitos funcionales describen las operaciones que debe realizar el sistema La última transmisión.

- **RF-01. Carga de historias:** el sistema deberá cargar historias narrativas desde archivos JSON independientes.
- **RF-02. Validación de historias:** el sistema deberá detectar errores estructurales, como escenas duplicadas, destinos inexistentes y variables no declaradas.
- **RF-03. Visualización de escenas y opciones:** el sistema deberá mostrar el texto de la escena actual y únicamente las opciones cuyas condiciones se cumplan.
- **RF-04. Procesamiento de decisiones:** el sistema deberá procesar las opciones seleccionadas, aplicar sus efectos sobre las variables y cambiar a la escena de destino correspondiente.
- **RF-05. Gestión del estado y los finales:** el sistema deberá conservar el estado de las variables durante la partida e identificar cuándo se alcanza una escena final.
- **RF-06. Función de deshacer:** el sistema deberá permitir deshacer la última decisión válida, restaurando la escena anterior y los valores previos de las variables. Una vez consumido el historial, otra solicitud no deberá modificar el estado.
- **RF-07. Reinicio de partida:** el sistema deberá permitir reiniciar la historia, regresar a la escena inicial, restaurar las variables iniciales y eliminar el historial de deshacer.
- **RF-08. Compatibilidad con diferentes historias:** el sistema deberá ejecutar la historia original y una segunda historia desde archivos JSON independientes, sin modificar específicamente el motor para cada una.

3.2. Requisitos no funcionales

Los requisitos no funcionales establecen criterios medibles para evaluar la calidad y el comportamiento del sistema.

- **RNF-01. Tiempo de respuesta:** el sistema deberá procesar una decisión válida y mostrar la siguiente escena en un máximo de un segundo en al menos el 95 % de 100 ejecuciones automatizadas, excluyendo el tiempo de escritura del usuario.
- **RNF-02. Detección de errores:** el validador deberá detectar el 100 % de los errores incluidos en un conjunto de pruebas previamente definido, que contemple destinos inexistentes, identificadores duplicados y variables no declaradas.
- **RNF-03. Compatibilidad de historias:** el motor deberá ejecutar correctamente dos archivos JSON independientes, incluida la historia original y una segunda historia de al menos tres escenas, sin modificaciones específicas en el código del motor.

Los valores establecidos para los requisitos no funcionales representan objetivos de aceptación. Su cumplimiento deberá comprobarse mediante pruebas y evidencias registradas; no se considerarán cumplidos hasta obtener los resultados correspondientes.

3.3. Preguntas, respuestas y supuestos

1. **¿Cómo se agregan nuevas historias?** Se asume que cada historia se almacena en un archivo JSON independiente que el motor puede cargar y ejecutar sin modificar su lógica principal.

2. **¿Qué debe recuperar la función de deshacer?** Se establece que debe restaurar la escena anterior y los valores de las variables previos a la última decisión válida. El historial se consume después de deshacer y se elimina al reiniciar la partida.

3. **¿Qué sucede si una historia contiene errores estructurales?** Se establece que el sistema debe informar de los errores detectados por el validador e impedir que la historia comience mientras estos errores estén presentes.

3.4. Revisión y corrección de requisitos

Durante la revisión de consistencia se identificó una ambigüedad en la formulación inicial: “El sistema debe permitir deshacer una decisión”. Esta expresión no especificaba qué información debía restaurarse ni qué sucedería si no existía una decisión anterior.

El requisito se corrigió como RF-06, estableciendo que el sistema debe deshacer únicamente la última decisión válida, restaurar la escena y las variables anteriores y no modificar el estado cuando el historial ya se haya consumido. Esta corrección permite verificar el comportamiento mediante pruebas específicas.

La revisión de los requisitos se complementará con una matriz de trazabilidad que relacione cada requisito funcional y no funcional con los componentes del sistema, los casos de prueba y las evidencias correspondientes.

4. Arquitectura

El proyecto La última transmisión utiliza una arquitectura organizada en componentes lógicos que separan el contenido narrativo, la lógica del sistema y la interfaz de interacción con el usuario. Esta organización permite mantener el código comprensible, facilitar las pruebas y agregar nuevas historias sin tener que modificar el motor narrativo.

La arquitectura se organiza en tres componentes principales:

- **Contenido narrativo:** corresponde a los archivos JSON que almacenan las escenas, las opciones, los destinos, las condiciones y los efectos de cada historia.
- **Motor narrativo:** implementado en Python, se encarga de cargar y validar las historias, administrar las variables de la sesión, evaluar las condiciones, aplicar los efectos, cambiar de escena, deshacer decisiones y reiniciar la partida.
- **Interfaz de usuario:** corresponde a la consola, mediante la cual el jugador visualiza las escenas, consulta las opciones disponibles e introduce sus decisiones. También muestra los mensajes de error y los resultados de la partida.

Los componentes se relacionan de manera que la interfaz recibe las decisiones del jugador y las comunica al motor narrativo. El motor procesa las reglas correspondientes y utiliza los datos de la historia para determinar el comportamiento de la partida. Finalmente, los resultados se presentan al usuario a través de la consola.

Esta separación facilita la incorporación de nuevas historias, debido a que el contenido se puede modificar mediante archivos JSON sin alterar las reglas del motor. Asimismo, permite que la interfaz pueda cambiarse en el futuro sin necesidad de reescribir las historias, siempre que se mantenga la comunicación con el motor narrativo.

4.1. Decisión arquitectónica 1: separar el contenido de la lógica del sistema

Contexto: el sistema debe ejecutar diferentes historias interactivas y permitir incorporar nuevas escenas, opciones, condiciones y efectos sin duplicar la lógica de programación.

Alternativas consideradas:

1. Almacenar las historias directamente en el código Python mediante estructuras y condiciones.
2. Almacenar las historias en archivos JSON y procesarlas mediante un motor narrativo común.

Elección: se eligió la segunda alternativa, utilizando archivos JSON para definir el contenido narrativo y Python para implementar las reglas de ejecución.

Consecuencias: esta decisión permite reutilizar el motor narrativo con diferentes historias, facilita la edición del contenido y reduce la duplicación de código. Como consecuencia, los archivos JSON deben mantener una estructura válida y cumplir las reglas de formato esperadas por el motor. Por ello, se incorpora un proceso de validación antes de iniciar la partida.

4.2. Decisión arquitectónica 2: utilizar una interfaz de consola

Contexto: el proyecto requiere una forma de interacción que permita ejecutar las historias, seleccionar opciones y comprobar las funcionalidades principales, manteniendo una implementación sencilla.

Alternativas consideradas:

1. Desarrollar una interfaz gráfica con botones y elementos visuales.
2. Utilizar una interfaz de consola basada en texto.

Elección: se eligió la interfaz de consola, debido a que permite concentrar el desarrollo en el motor narrativo, las reglas de las historias y las pruebas funcionales, sin añadir complejidad visual innecesaria.

Consecuencias: la solución es sencilla de ejecutar, facilita las pruebas automatizadas y reduce la cantidad de dependencias externas. Sin embargo, la experiencia de usuario es principalmente textual. En el futuro podría desarrollarse una interfaz gráfica reutilizando el motor narrativo y conservando los archivos JSON, siempre que se desacople adecuadamente la lógica de interacción.

4.3. Justificación de la separación entre contenido, reglas e interfaz

La separación entre contenido, reglas e interfaz responde al principio de responsabilidad única: cada componente debe tener una función claramente definida.

El contenido narrativo define qué sucede en la historia; el motor establece cómo se procesan las decisiones; y la interfaz determina cómo interactúa el jugador con el sistema. Esta organización reduce el acoplamiento, facilita el mantenimiento y permite probar la lógica sin depender exclusivamente de la interacción manual.

Además, la separación favorece la evolución del software, puesto que es posible agregar historias, corregir reglas o mejorar la interfaz de manera más controlada. De esta forma, se busca que los cambios en un componente tengan el menor impacto posible sobre los demás.

5. Implementación

La solución La última transmisión se implementó en Python, utilizando archivos JSON para almacenar el contenido de las historias y Git para el control de versiones. El programa permite cargar y validar historias, presentar escenas y opciones, procesar decisiones del jugador, administrar condiciones y efectos, deshacer decisiones anteriores y reiniciar la partida.

La implementación se organizó en los siguientes elementos:

- **`src/main.py`:** contiene la lógica principal del motor narrativo, la carga y validación de historias y la ejecución de las partidas.
- **`data/Historia.json`:** contiene la historia original del proyecto.

- **`data/segunda_historia.json`:** contiene una segunda historia que permite comprobar la reutilización del motor narrativo.
- **`tests/test_motor.py`:** contiene las pruebas automatizadas para verificar las funcionalidades y detectar errores o regresiones.
- **`docs/RF-06-deshacer.md`:** documenta el requisito relacionado con la funcionalidad de deshacer.
- **`docs/trazabilidad.md`:** relaciona los requisitos con los elementos de implementación y sus evidencias.
- **`README.md`:** documenta el propósito del proyecto y las instrucciones de instalación, ejecución y pruebas.

5.1. Pruebas y validación de la solución

Se utilizaron pruebas automatizadas mediante el módulo `unittest` de Python. Estas pruebas permiten comprobar funcionalidades importantes del motor narrativo y verificar que los cambios realizados no afecten comportamientos existentes.

Entre los casos de prueba considerados se encuentran:

- Validación de la historia original.
- Detección de destinos inexistentes.
- Detección de escenas duplicadas.
- Detección de variables no declaradas.
- Deshacer una decisión y restaurar el estado anterior.
- Comprobar que deshacer desde una situación sin historial no altere indebidamente la partida.
- Reiniciar la historia y eliminar el historial de deshacer.

También se verificó la ejecución de una segunda historia mediante el mismo motor narrativo, con el propósito de comprobar que el contenido puede cambiar sin tener que reescribir las reglas de ejecución.

Las pruebas se ejecutan mediante el siguiente comando:

```
python -m unittest discover -s tests -v
```

Para comprobar la sintaxis del programa se utiliza:

```
python -m py_compile src/main.py
```

Para ejecutar la historia original o la segunda historia se utilizan los siguientes comandos:

```
python src/main.py data/Historia.json
```

```
python src/main.py data/segunda_historia.json
```

La validación se complementa con la revisión de los archivos JSON y la comprobación manual de los recorridos narrativos. Los resultados y las evidencias de ejecución deben conservarse para demostrar el funcionamiento de la solución.

5.2. Control de versiones

Se utilizó Git para registrar la evolución del proyecto y GitHub como repositorio remoto. El historial de cambios permite identificar las funcionalidades incorporadas en cada versión y recuperar estados anteriores del código cuando sea necesario.

Durante el desarrollo se registraron cambios relacionados con la estructura inicial del proyecto, la implementación del motor narrativo, la validación de historias, las pruebas automatizadas y la funcionalidad de deshacer.

El control de versiones permite mantener evidencia de la evolución del software, identificar cuándo se introdujeron determinadas funcionalidades y facilitar la detección de regresiones. Para la entrega se debe incluir el historial de commits con sus autores, fechas y mensajes, junto con las evidencias de las versiones solicitadas por el docente.

5.3. Decisión de diseño detallado: separar los datos narrativos de las reglas de ejecución

Contexto: el sistema debe permitir ejecutar varias historias interactivas sin tener que desarrollar un motor diferente para cada una. Si las escenas, los destinos y las decisiones estuvieran escritos directamente en las instrucciones del programa, agregar una historia exigiría modificar el código y aumentaría el acoplamiento entre el contenido y la lógica.

Alternativas consideradas:

1. Definir cada historia directamente en el código Python mediante estructuras y condiciones específicas.
2. Almacenar las historias en archivos JSON y utilizar un motor común para interpretar sus datos.

Elección: se eligió almacenar el contenido narrativo en archivos JSON y mantener en Python las reglas que permiten cargar, validar y ejecutar las historias.

Consecuencias: esta decisión facilita reutilizar el motor, incorporar nuevas historias y modificar escenas sin alterar la lógica central. También permite probar la lógica con diferentes archivos de datos. Como contrapartida, el motor depende de que los archivos JSON respeten la estructura esperada, por lo que es necesario validarlos antes de comenzar la ejecución.

Esta decisión reduce el acoplamiento entre contenido y reglas, favorece el mantenimiento y facilita la ampliación del sistema.

5.4. Bibliotecas, herramientas y licencias

La implementación utiliza Python y sus módulos estándar para gestionar los archivos, interpretar los datos y ejecutar las pruebas. También se emplea Git para el control de versiones y GitHub para alojar el repositorio.

Biblioteca o herramienta	**Uso en el proyecto**	**Licencia**
Python	Lenguaje y entorno de ejecución.	PSF License.
`json`	Leer e interpretar las historias almacenadas en JSON.	Forma parte de la biblioteca estándar de Python; no requiere una instalación independiente.
`pathlib`	Construir y resolver rutas de archivos.	Forma parte de la biblioteca estándar de Python.
`unittest`	Ejecutar las pruebas automatizadas.	Forma parte de la biblioteca estándar de Python.
`sys`	Leer argumentos de la línea de comandos.	Forma parte de la biblioteca estándar de Python.
Git	Registrar cambios y administrar versiones.	GNU GPL v2.
GitHub	Alojar y consultar el repositorio remoto.	Servicio propietario; sus condiciones dependen del servicio y plan utilizado.